CSCI 2800 — Computer Architecture and Operating Systems (CAOS)
Lab 9 (document version 1.0)

- This lab is due by the **end of your lab session** on Wednesday, November 11, 2026
- **NEW** You must at least complete Checkpoint 1 in your Wednesday lab section to be allowed to have Checkpoints 2 or 3 checked off during office hours by the end of the week
- This lab is to be completed **individually**; do **not** share your code with anyone else
- Do **not** use AI to assist you; instead, use posted examples, the Discussion Forum, trial-and-error, and your own intelligence
- You **must** show your code and your solutions to a TA, CA, or mentor to receive credit for each checkpoint
- Note that all of your C code **must** successfully compile via `gcc` with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; also use `-Werror`, which treats all warnings as critical errors

For this lab, the goal is to continue to use the MIPS instructions and pseudo-instructions listed below. Remember that some instructions have "immediate" counterparts, e.g., `add` and `addi`; and the load and store instructions also have counterparts for different sizes (e.g., `lbu` and `sb`). Below, pseudo-instructions are marked with an asterisk `*`. Instructions/pseudo-instructions are: `add`; `and`; `beq`; `bne`; `j`; `jal`; `jr`; `la*`; `li*`; `lw`; `move*`; `or`; `sll`; `slt`; `srl`; `sw`; and `sub`.

For any of the above instructions that you do not use for this lab, make sure you understand and practice using them elsewhere!

1. **Checkpoint 1:** For the first checkpoint, using MIPS, write a function labeled `strlen` that counts the number of characters in the string starting at the memory address specified in `$a0`. Use register `$v0` as the return value for your function—i.e., return the string length in `$v0`. Test your function on a variety of strings, including an empty string.

   Next, write a function labeled `wordtri` that prints the string starting at the memory address in `$a0` as a triangle. Use your `strlen` function to first determine the length of the string. A few examples are shown below; in each case, the original word is displayed first.

   Do **not** modify the original string, i.e., the string `$a0` is essentially passed in as a `const` pointer. Therefore, consider also writing a `strcpy` function. You will need to use the `sbrk` system call to dynamically allocate memory—see the in-class notes from October 20 for more details.

```
ABRACADABRA          CAOS          GOLDSCHMIDT
 BRACADABR            AO            OLDSCHMID
  RACADAB                           LDSCHMI
   ACADA                             DSCHM
    CAD                               SCH
     A                                 C

RENSSELAER POLYTECHNIC          123$%^&*(0-=_+<>,.
 ENSSELAER POLYTECHNI            23$%^&*(0-=_+<>,
  NSSELAER POLYTECHN              3$%^&*(0-=_+<>
   SSELAER POLYTECH                $%^&*(0-=_+<
    SELAER POLYTEC                  %^&*(0-=_+
     ELAER POLYTE                    ^&*(0-=_
      LAER POLYT                      &*(0-=
       AER POLY                        *(0-
        ER POL                          (0
         R PO
           P
```

   Be sure that for your function calls, you preserve all saved registers `$s0-$s7`, the stack pointer `$sp`, and the return address `$ra`, as indicated on the MIPS Reference Card.

   Be fully prepared to explain what each MIPS instruction in your code does.

2. **Checkpoint 2:** For the second checkpoint, write a function labeled `checksum` that calculates the "Internet checksum" of the data starting at the memory address in `$a0`. The length of the data (in bytes) is passed in using register `$a1`. And the calculated checksum is returned in register `$v0`.

   To calculate the Internet checksum, also referred to as the "one's complement sum," sum all 16-bit halfwords starting at `$a0`. If there is odd number of bytes, the last byte must be extended to 16 bits, e.g., `0x49` is extended to `0x0049` (and is the last 16-bit value added).

   Whenever your sum exceeds 16 bits, i.e., a carry bit is present as the 17th bit, simply add the carry to the accumulated sum. For example, if `0xf123` is added to `0xf000`, the sum is `0x1e123`—therefore, to stay within 16 bits, add the carry bit to `0xe123` to obtain `0xe124`.

   The final step of calculating the Internet checksum is to flip all of the bits. For example, if the resultant sum is `0xe123`, you would apply a logical bitwise NOT operator to obtain `0x1edc` as the checksum. (To see this, write these values in binary.)

   To test your function, step through a few example inputs and display the hexadecimal contents of `$v0` in `spim` via the `print_all_regs hex` or `print $N` commands.

   As an example, input string `"ABCDEFGHI"` produces an Internet checksum of `0xeea1`, calculated as follows:

```
"ABCDEFGHI" ==> 0x4142434445464748

  0x4142
  0x4344
  0x4546
  0x4748
+ 0x0049
--------
 0x1115d  <== carry bit

  0x115d
+ 0x0001
--------
  0x115e  ==> flip all bits to obtain 0xeea1
```

   Be sure that for your function calls, you preserve all saved registers `$s0-$s7`, the stack pointer `$sp`, and the return address `$ra`, as indicated on the MIPS Reference Card.

   Be fully prepared to explain what each MIPS instruction in your code does.

3. **Checkpoint 3: At the beginning of your Wednesday lab section, this lab checkpoint will be available and contain further instructions.** As a hint, keep working with the MIPS instruction set; this checkpoint will extend Checkpoint 2.